Location Services: Documentation

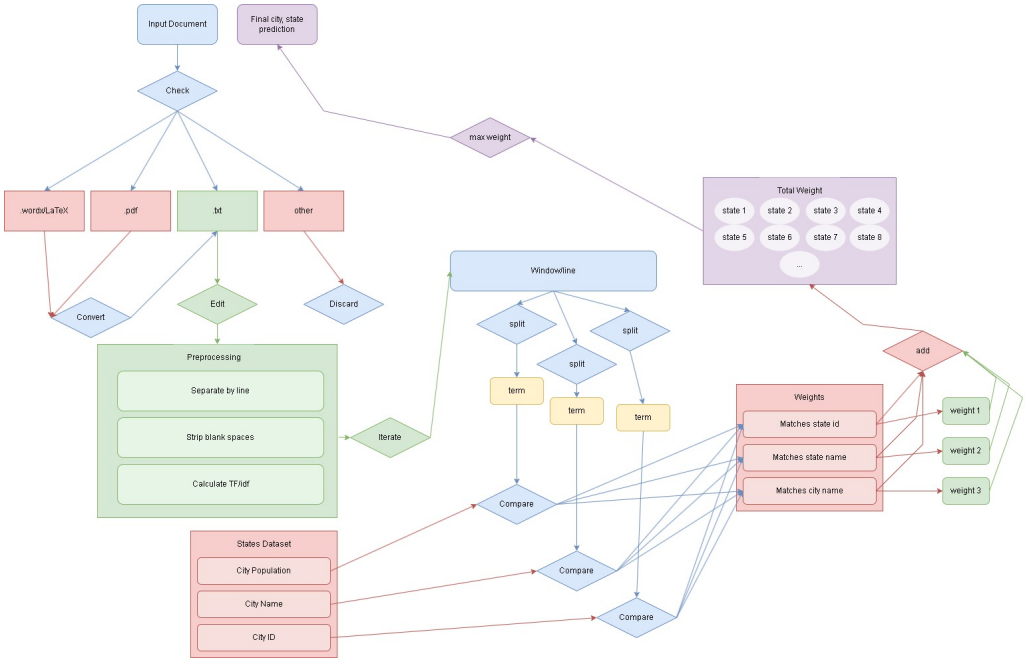
Acey Dufresne[1*] [1]Doves Network, 2026

OVERVIEW

The location services included in the complete algorithm architecture are an optional service for users that do not enter their location into whichever medium the final algorithm is used on. Three inputs are passed to the algorithm: user's name(optional), user's resume file, and the user's address(optional). The most best case, most efficient use of the algorithm is to have all three inputs, however, one must always assume the worst case scenario when designing solutions. In this case, specifically in regards to location vectors, that the user does not enter an address.

If the user does not provide an address this sub-algorithm/function will make a prediction on the best location option. It is important to note, this sub function will not be ran for every user, only when a location is missing. The user uploads their resume, the site/application/other medium used to host the algorithm, passes the file input as well as an empty value. When this value is detected, the algorithm makes a call for the location services.

The main class, or node, which acts as the coordinator or director for the algorithm, which is the class that physically runs the different sections of the algorithm, checks what format the file input is in. If it is a .wordx extension, Microsoft's Word files, then the location service function is given that information, likewise the same is true if the extension is either ".pdf," or ".txt." The difference is minimal between them, besides the fact that the step of converting the file to text can be disregarded in the case of a .txt file, which is the least file format for resumes. Then the function can compare each term of a window to the data set, applying weights for certain terms. In the end the highest weighed term is predicted. The other location services are gathered from the same dataset that is used from the input function. This service focuses on finding the closest corresponding city, listed on the job boards that will be searched later. For example, LinkedIn only uses nine city locations within the continental United States, these are the cities that users can sort by, and likewise, the algorithm can utilize. The algorithm uses Euclidean distance, gathered from the longitude and latitude from the corresponding dataset, and calculates the closest listed city from each job board application.



]

1. VECTOR PREDICTION

1.1. Dataset/Pandas:

Pandas: The Python Module used to read in CSV files, specifically for the data in the chosen set. Heavily used in each aspect of the algorithm, with limited to no strain on efficiency or resources. US Cities.csv: The dataset previously mentioned in ”Web-Crawler: Overview Architecture,” documentation. Used for multiple steps. The primary columns needed are the: City names, State Names, State ID’s, and state population. Each corresponds to a unique weight. Using one dataset also ensures resources for the algorithm are being used in a thoughtful matter, and no additional operations are needed, aside from reading in the dataset once, when the location services are called.

2. USER WALKTHROUGH:

After the input file has been converted to text format, as a String type array, that array is iterated by Window. The window, or each line from the original input is then ran through the pre-processing steps:

2.1. Pre-processing:

The csv dataset is read in, and four columns are used to create four map sets. These would be the state names, state id’s, city names, and state populations. Each has a corresponding weight bias which will be used for the final prediction.

The term pre-processing, is slightly deceptive, in this specific case. The pre-processing happens in several separate instances as the function runs. Initially, the only steps done, are to split each window, where the original Window array, is now an array of terms. In addition, the punctuation is removed from each window. Once the input is checked in its entirety, for only state id’s, for example: ”GA,” ”CA,” and ”MI,” the matches are recorded. These matches carry a heavy bias weight, as they are very specific.

After this has been done, the text is capitalized, this ensures a higher likelihood of matches, despite improper grammar, within the algorithm. Then city names and state names may be checked, giving us the final pre-processing required for these functions.

2.2. Weights and Biases:

After pre-processing has been conducted, each individual term is ran through each seperate dataset, the matches are kept track of. Each set has a different weight, for example, city names and state names are weighted the same, as the algorithm will return two predictions, both the city and state’s name of the user, however, the population bias is much less, and used generally for edge cases, if two locations have an equal probability, then the population weight, whichever state has the larger population, will determine the final prediction.

For example, two options are weighted the same, the formula used to determine the population bias: (Current Option’s Population/Maximum US City Population) * .2. This function returns a relatively, compared to the rest of the bias weights, low value, which is then added to both predictions. This can be related to the final output layers of an ANN, which has edge weights.

Additionally, the algorithm needs the ability to return a NULL value, or empty. This is important for the accuracy and precision results, specifically, if none of the predictions have a high enough value, we do not want to return a poor prediction, better to have no prediction, than to influence the overall algorithm. If the result is not an empty value, the algorithm then needs to commit a sanity check. This involves ensuring state and city names match, for example if the highest prediction state is ”California,” or ”CA,” and the highest weighed city name is ”Atlanta,” the algorithm needs to catch this error.

For this specific algorithm, the heavier bias is kept. Each prediction is sorted into a stack, with the heaviest, or best matches, towards the top of the stack. In the preposed example, if ”CA,” has a weight of .6, and ”Atlanta,” a weight of ”.4,” then the city prediction is ”popped” off the stack, and the next highest prediction is used. In this way, we keeping exchanging the prediction, until either: a) both the ”heads” of the state and city prediction match, or b) the lower weighed prediction becomes heavier than the initial prediction(this should never occur).

2.3. Safeguards and Sanity:

As briefly discussed above, there are some sanity checks and safety bumpers that must be considered. For example, as can be seen in the results section, initially many of the incorrect predictions came from common words, that happen to also be cities. For example, ”lead,” ”Commerce,” and ”University.” The algorithm handles some forms of this with it’s pre-processing, ensuring ’in,’ isn’t conflicted with ”IN,” but further steps are required to achieve the final results.

To this end, the tf-IDF score is implemented to account for the ’commonality,’ of terms. That is TF = appearance of terms in the input / total amound of terms in the input; IDF = log(amount of documents in the dataset / 1 + appearance of terms in the input);

tf-IDF = IDF * tf. Using this, the algorithm can apply a smaller weight to terms that are more common in the dataset.

With all this put together, the algorithm can reasonably determine a safe prediction, either NULL, or some value, for any resume. Additionally, a regex pattern should be implemented for zip codes. This would include implementing a new dataset of all zip codes in the US, a hash map to map values to codes, all of which introduces additional operations, and in turn costs the algorithm resources in terms of efficiency. This must be agreed upon, after seeing the results, whether additional precision is required for this use case.

Another safe guard that should be considered, is to include proper nouns. A high construct example of this would be to account for companies, trade, and universities. However, in the pursuit of speed and efficiency, this algorithm, in this facet, should only account for Universities. For example, a high level model, may take in account a company, then determine that the corporation is operational in multiple states, then could use a very low level weight to determine what states that trade is most common and apply that weight to the final prediction. However, this begins to astray into a deeper learning mechanism, which this algorithm is actively attempting to avoid, again for this use case and speed.

Instead, let the algorithm only account for higher education, due to how federal funding works, a University, usually, cannot exist in more than one state. Now assign a low level weight to these values, as according to the statistics, it becomes more uncommon for an individual to stay in the same state that they achieved their degree in, as they get older. In this way, the hueristic approach is kept intact, and furthermore, these biases are used as edge cases.

3. LOCATION EMBEDDING:

3.1. Overview:

There are pre-existing Python modules for locational vector embeds, however for this specific use case, the algorithm should also take other factors into account. The proposed method, is a custom vector embedding process, which uses Haversine distance. Specifically to make use of the datasets that already in use in the model, this would cut down on additional operations. Using the longitude and latitude coordinates as the base vectors.

3.2. Implementation:

Using the pre-existing dataset to gather the base vectors of the user inputted cities, would reduce the tax on resources. Furthermore, using a sourced module or library that handles geo-vectors, would be much more computationally expensive, as downloading a model for this use case would require the model keeping track of all city to coordinate elements. In contrast, with the proposed methodology, the algorithm would only need to keep track of the main cities that the job boards have listed, for this moment estimated to be around 15 cities total, plus one additional city, which is the predicted city the user inputted.

This also gives the benefit of directly applying operational tasks to these vectors. For example, consider the algorithm taking into account not only the Haversine distance, but estimated distance of roads and highways between cities. These road lengths will again be used as an edge weight, which would be substantially more difficult using a pre-formed geo-embedding library.